

Sujet fil rouge – Battle Royal

Introduction

L'objectif de ce projet est de créer au fil des séances un serveur Java REST permettant de jouer à un jeu Battle Royal simplifié

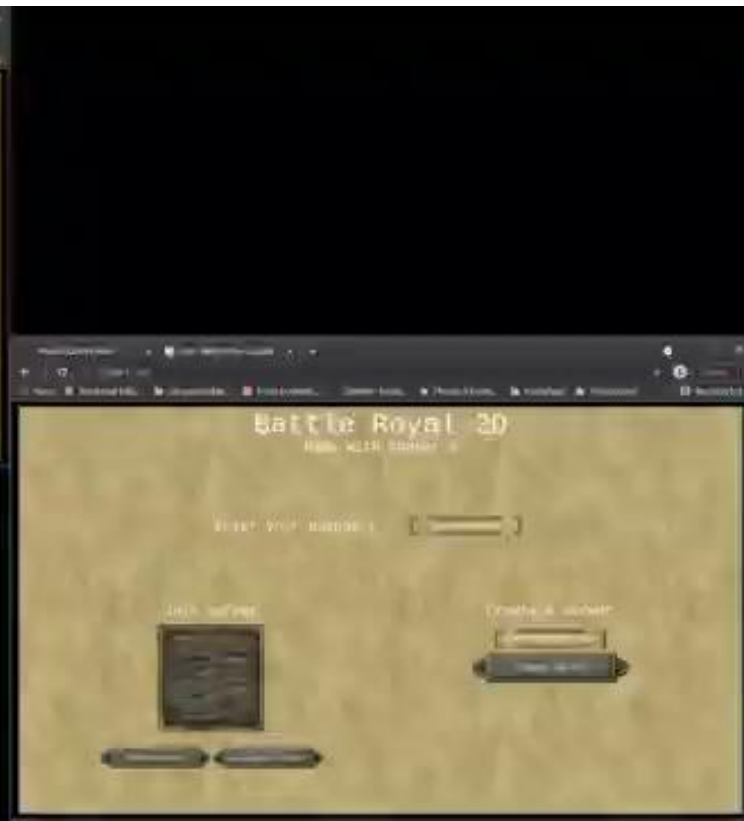
Le front end du jeu ainsi que le contrat d'interface vous sont fournis. L'objectif est de réaliser le serveur backend en Java.

Organisation des séances

Date	Objectif de la séance
04/06 (1h30)	● Conception du serveur ● Implémentation de la mécanique de création de partie et de listing des parties existantes.
02/07 (1h30)	● Implémentation de la création et de destruction de joueurs. ● Implémentation de la mise à jour des positions des joueurs.
29/08 (1h30)	● Implémentation du game status, gestion du cycle du jeu. ● Implémentation du broadcast les bullets
Bonus (3h)	● Implémentation de la création de bullets ● Implémentation de la destruction des bullets

Présentation du projet

Démonstration du résultat final



Présentation des interfaces

A l'affichage du menu et au démarrage d'une partie, le front end crée un websocket.

Toutes les interactions entre le front et le back ont lieu via ce websocket avec des payloads en json.

Dans les slides suivants, le code couleur ci-dessous est utilisé:

- Message du client vers le serveur
- Message du serveur vers le client
- Message du serveur vers tous les autres clients (broadcast)

Informations générales

Message type

```
export enum MessageType {  
    NEW_PLAYER    = 1,  
    PLAYER_MOVED  = 2,  
    GAME_STATE    = 3,  
    PLAYER_DESTROY = 4,  
    NEW_BULLET    = 5,  
    BULLET_DESTROY = 6,  
    NEW_GAME       = 7,  
    LIST_GAME      = 8,  
};
```

Puisque toutes les communications front/back passent via un websocket, le message type permet d'identifier le type de messages que l'on est en train de traiter.

Par ailleurs tous les messages sont accompagnés d'un websocket ID.

Informations générales

GameStatus

```
export enum GameStatus {  
    LOBBY = 1,  
    STARTING = 2,  
    PLAYING = 3,  
    FINISHED = 4  
};
```

L'enum GameStatus permet de savoir dans quel état se trouve la partie à un instant t. Elle est présente dans le game state.

Création de parties

```
{  
  "socketUuid":  
"42bbd929-84d9-42a5-bf1a-ca917f9ef59e",  
  "type": 7,  
  "gameId":  
"0f74aa9c-298d-45ac-aa26-a189ca00ac53",  
  "gameName": "Game1"  
}
```

Lorsque l'utilisateur crée une partie depuis le front, le message json suivant est envoyé via le websocket.

Le backend doit alors créer une partie en lui attribuant le bon nom et le bon uuid.

Si une erreur survient, elle doit être indiquée en réponse avec le status KO, sinon une réponse est émise en status OK avec un "errorMessage" vide.

```
{  
  "status": "OK",  
  "errorMessage": ""  
}
```

Listing de parties

```
{  
  "socketUuid":  
    "966f369c-bf9e-42e0-bf33-a3aa95ac52ce",  
  "type": 8  
}
```

Lors du chargement du menu ou lorsque l'utilisateur clique sur le bouton "Refresh server list".

Le serveur doit alors fournir la liste des serveurs disponibles quelque soit l'état de la partie (des joueurs peuvent rejoindre la partie en cours de route) en suivant l'interface ci-contre.

```
{  
  "gamelist": [  
    {  
      "gameId":  
        "17a6b09a-c598-4999-81dd-05bb2f9ce7f3",  
      "gameName": "Game1"  
    },  
    ...  
  ],  
  "type": 8  
}
```

Création de joueurs – Request

```
{
  "socketUuid":
"25ebd3e1-d744-4cd6-aed7-370c8762c275",
  "type": 1,
  "player": {
    "uuid":
"4a5571b6-20bb-48f4-a3aa-858efcae403d",
    "name": "Player1",
    "x": 1434.666666666667,
    "y": 264,
    "velocityX": 0,
    "velocityY": 0,
    "direction": 0,
    "atlas": "misa",
    "frame": "misa-back",
    "width": 30,
    "height": 40
  },
  "gameId":
"3830f6de-d2d5-4829-a4f9-c42af561bb8d"
}
```

Lorsqu'un joueur rejoint une partie (qu'il en soit le créateur ou non), le message suivant est émis par le front.

Le back doit alors retrouver la partie concernée, et ajouter le joueur et toutes ses caractéristiques dans la partie.

Chaque paramètre est important, il devra être stocké puis renvoyer aux autres joueurs pour réaliser le rendu.

Création de joueurs – Response with error

```
{  
  "status": "KO",  
  "errorMessage": "A player with the same uuid  
already exists in this game."  
}
```

Si une erreur a lieu, un message de type KO est envoyé.

Création de joueurs – Response

```
{
  "type": 3,
  "players": [
    {
      "uuid":
"f2cbcc5a-32f2-4e50-b3a0-2acce394cc52",
      "name": "Player2",
      "x": 1434,
      "y": 264,
      "velocityX": 0,
      "velocityY": 0,
      "direction": 0,
      "atlas": "misa",
      "frame": "misa-back",
      "width": 30,
      "height": 40,
      "isAlive": true
    }
  ],
  "gameStatus": 1,
  "winnerName": ""
}
```

S'il n'y pas d'erreur, le back doit répondre avec un game state. Un game state contient toutes les informations concernant la partie et les **autres** joueurs.

Grâce à ces informations, le front va réaliser le rendu graphique des autres joueurs.

Mis à jour d'un joueur – Request

```
{
  "socketUuid":
"bfa77ee8-aaad-4311-822d-9c93cb980520",
  "gameId":
"fb45ff34-412b-40ff-ae32-40d841f5bd96",
  "type": 2,
  "player": {
    "uuid":
"95bdc5b9-e78f-483d-9c3b-05eb3a71abed",
    "name": "Player1",
    "x": 1434.666666666667,
    "y": 264,
    "velocityX": 0,
    "velocityY": 0,
    "direction": 0,
    "atlas": "misa",
    "frame": "misa-back",
    "width": 30,
    "height": 40,
    "isAlive": true
  }
}
```

Toutes les X millisecondes, le client envoie au serveur les informations de son joueur selon l'interface ci-contre.

Mise à jour d'un joueur – Response with error

```
{  
  "status": "KO",  
  "errorMessage": "The player could not be  
updated: game not found."  
}
```

Si une erreur a lieu, un message de type KO est envoyé.

Création de joueurs – Response

```
{  
  "type": 3,  
  "players": [  
    {  
      "uuid":  
"f2cbcc5a-32f2-4e50-b3a0-2acce394cc52",  
      "name": "Player2",  
      "x": 1434,  
      "y": 264,  
      "velocityX": 0,  
      "velocityY": 0,  
      "direction": 0,  
      "atlas": "misa",  
      "frame": "misa-back",  
      "width": 30,  
      "height": 40,  
      "isAlive": true  
    }  
  ],  
  "gameStatus": 1,  
  "winnerName": ""  
}
```

S'il n'y pas d'erreur, le back doit répondre avec un game state selon le même protocole que lors de la création d'un joueur.

Mis à jour du Game Status

```
{  
  "type": 3,  
  "gameStatus": 3,  
  "gameId":  
  "7367b868-7e75-4de0-8ea7-704c2583e2a1"  
}
```

```
{  
  "status": "OK",  
  "errorMessage": ""  
}
```

Le game status évolue au fil de la partie. Au début il est initialisé à LOBBY.

Lorsque le créateur de la partie clique sur START, le statut passe à l'état STARTING via un message émis par le front. Les joueurs sont alors répartis sur la carte mais ne peuvent ni tirer ni se déplacer.

Au bout de 3 secondes, le statut passe à l'état PLAYING via un message émis par le front. Les joueurs jouent librement.

Mis à jour du Game Status

Fin de partie

```
{
  "type": 3,
  "players": [
    {
      "uuid":
"f2cbcc5a-32f2-4e50-b3a0-2acce394cc52",
      "name": "Player2",
      "x": 1434,
      "y": 264,
      "velocityX": 0,
      "velocityY": 0,
      "direction": 0,
      "atlas": "misa",
      "frame": "misa-back",
      "width": 30,
      "height": 40,
      "isAlive": true
    }
  ],
  "gameStatus": 4,
  "winnerName": ""
}
```

Le serveur est en charge d'identifier quand la partie est terminée.

A chaque mis à jour des joueurs, le serveur doit vérifier le nombre de joueur toujours en vie. Si ce nombre de joueur vaut 1, le serveur passe automatiquement le status dans l'état finished.

Cette information est mise à jour dans le game state renvoyé à tous les joueurs. Le front va alors téléporter tous les joueurs dans le lobby.

Gestion des bullets – Création

```
{
  "socketUuid": "3825032e-a5a4-4868-8f44-f6f402affcb8",
  "type": 5,
  "bullet": {
    "uuid": "139bbe56-2fc8-4d14-b026-b0562d9c73dd",
    "startX": 1289.6666666666664,
    "startY": 2360,
    "toX": 1475,
    "toY": 2392
  },
  "gameId": "df68fadb-4549-41f1-b95b-17efa576a75f"
}
```

Le mécanisme des bullets est indépendant du game state.

Lorsqu'une bullet est émise par un joueur, le serveur doit envoyer cette information à tous les autres joueurs de la partie via un système de broadcast à implémenter.

Le client émetteur de la balle aura pour réponse un status de type OK ou KO.

```
{
  "type": 5,
  "gameId": "b8da6a8f-21fc-48bc-b1ac-8c9309685d4e",
  "bullet": {
    "uuid": "9051df74-8776-4980-a5a8-85f6af5bbe81",
    "startX": 1019,
    "startY": 2570,
    "toX": 951,
    "toY": 2413
  }
}
```

Gestion des bullets – Destruction

```
{  
  "socketUuid": "5cadfcfe-6a49-42cc-b036-edddda523ccb",  
  "type": 6,  
  "bullet": {  
    "uuid": "3bdf567a-2492-4cac-9892-af8c3222a382",  
    "startX": 913,  
    "startY": 2269,  
    "toX": 919,  
    "toY": 2177  
  },  
  "gameId": "76a25466-8a15-418a-9834-7f9096e4301a"  
}
```

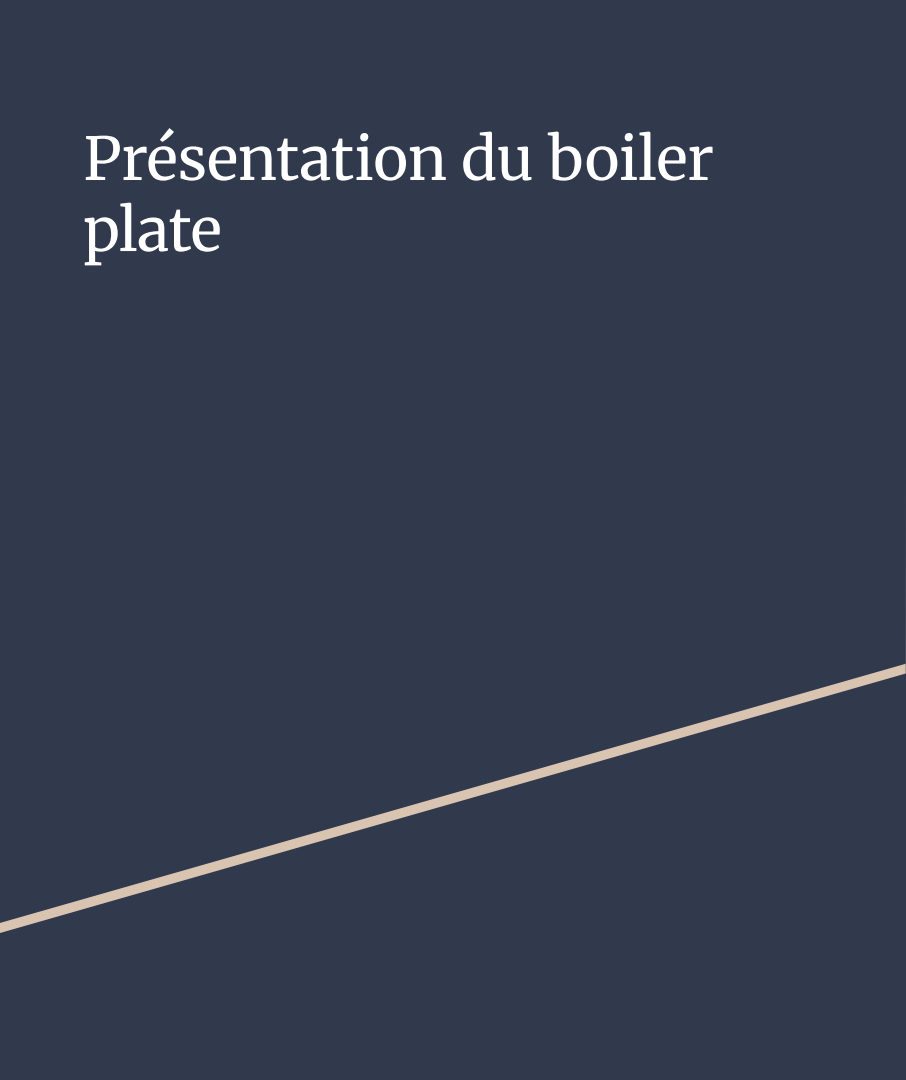
Lorsqu'une balle rencontre un mur ou un autre joueur, elle doit être détruite.

Le client émet alors un message de destruction qui sera lui aussi broadcast à tous les autres clients présents dans la partie.

Le client émetteur du message aura en retour en message de status OK ou KO.

```
{  
  "type": 6,  
  "gameId": "76a25466-8a15-418a-9834-7f9096e4301a",  
  "bullet": {  
    "uuid": "3bdf567a-2492-4cac-9892-af8c3222a382",  
    "startX": 913,  
    "startY": 2269,  
    "toX": 919,  
    "toY": 2177  
  }  
}
```

Présentation du boiler plate



Conception du serveur

Proposition d'UML

Handlers

SocketHandler

```

+ static void handleNewConnection(WsConnectContext, BroadcastService)
+ static void handleCloseConnection(WsCloseContext)
+ static void handleOnMessage(WsMessageContext, GameService, BroadcastService)
- void createPlayerHandler(WsMessageContext, GameService, BroadcastService)
- void deletePlayerHandler(WsMessageContext, GameService, BroadcastService)
- void playerMovedHandler(WsMessageContext, GameService, BroadcastService)
- void newBulletHandler(WsMessageContext, GameService, BroadcastService)
- void
    
```

DTO

MessageType

WebSocketMessage

```

+ type : MessageType
+ socketUUID : uuid
    
```

ResponseMessage

BulletDestroyMessage

DestroyBulletResponse

DestroyPlayerMessage

GameStateMessage

GameStateMessage

GameStateResponse

ListGameResponse

NewBulletMessage

NewBulletResponse

NewGameMessage

NewPlayerMessage

PlayerMovedMessage

Services

BroadcastService

```

+ uuid : UUID
+ startX : int
+ toX : int
+ toY : int

+ void addNewWebsocket(UUID, UUID, WsMessageContext)
+ void removeWebSocket(UUID, UUID, WsMessageContext)
+ void broadcastMessageInGame(UUID gameUuid, String message)
+ void broadcastMessageInGameExcludeCurrent(String, UUID, String)
    
```

GameService

```

- gameManager : IGameManager

+ createGameService(NewGameMessage)
+ Optional<GameStateResponse> createPlayerService(NewPlayerMessage)
+ Optional<GameStateResponse> deletePlayerService(DestroyPlayerMessage)
+ Optional<GameStateResponse> updatePlayerService(PlayerMovedMessage)
+ Optional<NewBulletResponse> createBulletService(NewBulletMessage)
+ Optional<DestroyBulletResponse> destroyBulletService(BulletDestroyMessage)
+ ListGameResponse listGameService()
+ Optional<GameStateResponse> updateGameStatus(GameStateMessage)
    
```

Interfaces

IGameManager

Game

GameManager

```

+ games : Map<UUID, GameSession>

+void createGame(UUID, String)
+ GameState createPlayer(UUID, Player)
+ GameState destroyPlayer(UUID, Player)
+ GameState playerUpdate(UUID, Player)
+ void createBullet(UUID, Bullet)
+ void destroyBullet(UUID, Bullet)
+ GameState setGameState(UUID, GameStatus)
+ ArrayList<Game> getGameList()
- GameSession getGameFromId(UUID)
    
```

GameSession

```

- gameUUID : UUID
- gameName : String
- players : Set<Player>
- bullet : ArrayList<Bullet>
- status : GameStatus
- winnerName : Optional<String>

+ void createPlayer(Player)
+ void deletePlayer(Player)
+ void updatePlayer(Player)
+ void newBullet(Bullet)
+ void destroyBullet(Bullet)
+ Set<Player> getOtherPlayers(UUID)
+ Set<Player> getPlayers()
+ Optional<String> getWinnerName()
+ GameStatus getGameStatus()
+ void setGameStatus(GameStatus)
- void isGameFinished()
    
```

Models

Comparable

Bullet

```

+ uuid : UUID
+ startX : int
+ toX : int
+ toY : int

+ velocityX : int
+ velocityY : int
+ direction : int
+ atlas : String
+ frame : String
+ width : int
+ height : int
+ isAlive : bool

+ compareTo(Bullet); int
+ equals(Object) : bool
    
```

Player

```

+ uuid : UUID
+ name : String
+ x : int
+ y : int
+ velocityX : int
+ velocityY : int
+ direction : int
+ atlas : String
+ frame : String
+ width : int
+ height : int
+ isAlive : bool

+ Player(Player)
+ compareTo(Player); int
+ equals(Object) : bool
    
```

GameState

```

+ players : Set<Player>
+ gameStatus : GameStatus
+ winnerName : Optional<String>
    
```

Game

```

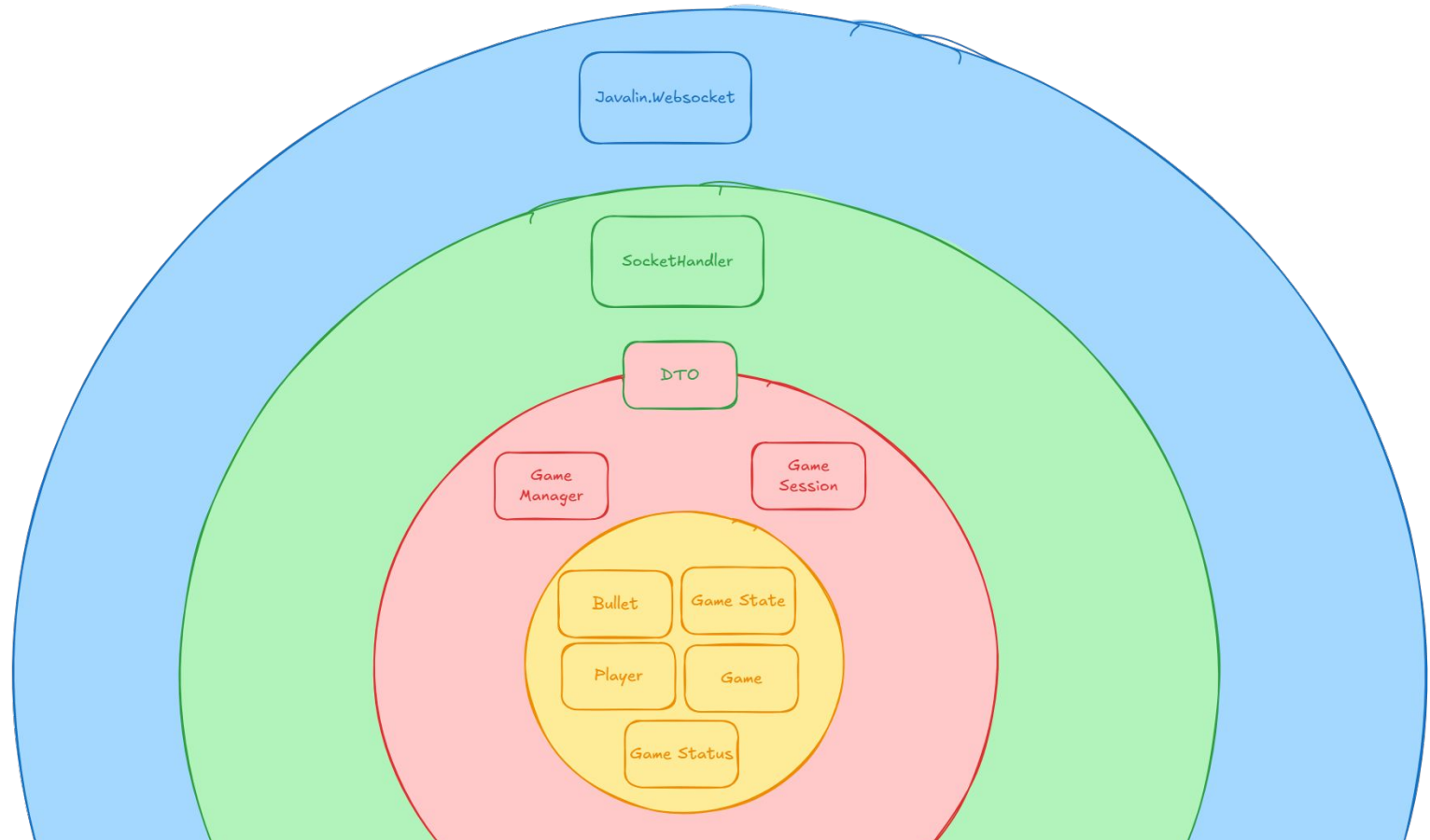
+ uuid : UUID
+ name : String
    
```

GameStatus

```

IDLE = 0
LOBBY = 1
STARTING = 2
PLAYING = 3
FINISHED = 4
    
```

Clear Archi Représentation



Code snippet

Code Snippet – Serialize/Deserialize data with Javalin

Deserialize

```
NewPlayerMessage newPlayerMessage = ctx.message(NewPlayerMessage.class);
```

Serialize

```
public class GameStateResponse {  
    public final Integer type = MessageType.GAME_STATE.type;  
    public Set<Player> players;  
    public GameStatus gameStatus;  
    public String winnerName = "";  
}  
  
GameStateResponse state = new GameStateResponse();  
ctx.send(state.get());
```

Code Snippet – Serialize data with fasterxml jackson

Serialize

```
import com.fasterxml.jackson.databind.ObjectMapper;

try {
    ObjectMapper Obj = new ObjectMapper();
    String jsonStr = Obj.writeValueAsString(stateResponse.get());
} catch (JsonProcessingException e) {
    logger.error(e.getMessage());
}
```

A vous de coder !